

# Docker Lab Submission

**Name:** Ayush Patel

**Roll No:** 202301084

All steps below were performed on a Linux machine (user ayushpatel) with Docker Engine 29.6.2 and Docker Compose v5.3.1. Every screenshot is the real output of the commands shown above it.

# Project Files

---

## Dockerfile

```
# Base image: lightweight Nginx web server on Alpine Linux
FROM nginx:alpine

# Metadata about the image author
LABEL maintainer="Ayush Patel (202301084)"

# Copy our web page into Nginx's default web root
COPY app/index.html /usr/share/nginx/html/index.html

# Document that the container listens on port 80
EXPOSE 80

# Start Nginx in the foreground so the container keeps running
CMD ["nginx", "-g", "daemon off;"]
```

## docker-compose.yml

```
services:
  web:
    build: .
    image: ayushpatel/docker-lab-web:1.0
    container_name: ayushpatel-compose-web
    ports:
      - "8081:80"
    volumes:
      - web-logs:/var/log/nginx
    restart: unless-stopped

volumes:
  web-logs:
```

## app/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Docker Lab - Ayush Patel</title>
  <style>
    body { font-family: Arial, sans-serif; background: #0d47a1; color: #fff;
      display: flex; align-items: center; justify-content: center; height: 100vh; margin: 0; }
    .card { background: #fff; color: #222; padding: 40px 60px; border-radius: 12px; text-align: center; }
    h1 { color: #0d47a1; margin-top: 0; }
  </style>
</head>
<body>
  <div class="card">
    <h1>Hello from Docker!</h1>
    <p><b>Name:</b> Ayush Patel</p>
    <p><b>Roll No:</b> 202301084</p>
    <p>This page is served by an Nginx container built from my own Dockerfile.</p>
  </div>
</body>
</html>
```

# Step 1: Check Docker installation

| Command                                                                                                                                       | Purpose                                                                                                                        |
|-----------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <code>docker --version</code>                                                                                                                 | Shows the installed Docker client version, confirming Docker is installed.                                                     |
| <code>docker compose version</code>                                                                                                           | Shows the Docker Compose plugin version (used later for multi-container deployment).                                           |
| <code>docker info --format 'Server: {{.ServerVersion}}   Containers: {{.Containers}}   Images: {{.Images}}   OS: {{.OperatingSystem}}'</code> | Prints key details about the Docker daemon (engine version, number of containers/images, OS) to confirm the daemon is running. |

```
ayushpatel@docker-lab: ~/docker-lab  -  Step 1: Check Docker installation

ayushpatel@docker-lab:~/docker-lab$ docker --version
Docker version 29.6.2, build dfc4efb
ayushpatel@docker-lab:~/docker-lab$ docker compose version
Docker Compose version v5.3.1
ayushpatel@docker-lab:~/docker-lab$ docker info --format 'Server: {{.ServerVersion}} | Containers: {{.Containers}} | Images: \
> {{.Images}} | OS: {{.OperatingSystem}}'
Server: 29.6.2 | Containers: 0 | Images: 0 | OS: Ubuntu 24.04.4 LTS
ayushpatel@docker-lab:~/docker-lab$
```

## Step 2: Run the first container (hello-world)

| Command                             | Purpose                                                                                                                                                                |
|-------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>docker run hello-world</code> | Downloads the hello-world image (if not present), creates a container from it and runs it. The container prints a message and exits - proving Docker works end to end. |

```
ayushpatel@docker-lab: ~/docker-lab - Step 2: Run the first container (hello-world)

ayushpatel@docker-lab:~/docker-lab$ docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pulling fs layer
4f55086f7dd0: Download complete
d5e71e642bf5: Download complete
4f55086f7dd0: Pull complete
Digest: sha256:5e23090353324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8
Status: Downloaded newer image for hello-world:latest
ayushpatel@docker-lab:~/docker-lab$
```

### Step 3: Pull images and list them

| Command                                | Purpose                                                                               |
|----------------------------------------|---------------------------------------------------------------------------------------|
| <code>docker pull nginx:alpine</code>  | Downloads the official Nginx web-server image (small Alpine variant) from Docker Hub. |
| <code>docker pull alpine:latest</code> | Downloads the tiny Alpine Linux base image.                                           |
| <code>docker images</code>             | Lists all images stored locally with their repository, tag, ID and size.              |

```
ayushpatel@docker-lab: ~/docker-lab - Step 3: Pull images and list them

ayushpatel@docker-lab:~/docker-lab$ docker pull nginx:alpine
alpine: Pulling from library/nginx
e72112c14215: Pulling fs layer
745dfb2690dd: Pulling fs layer
e76228b47809: Pulling fs layer
9a9a644fdd6a: Pulling fs layer
d9aae54b5831: Pulling fs layer
6c53d0b2a666: Pulling fs layer
e2de96513ba9: Pulling fs layer
64c8194480fe: Pulling fs layer
745dfb2690dd: Download complete
64c8194480fe: Download complete
d54d3939625e: Download complete
9a9a644fdd6a: Download complete
6c53d0b2a666: Download complete
0deea27e58d9: Download complete
e76228b47809: Download complete
e2de96513ba9: Download complete
e72112c14215: Download complete
d9aae54b5831: Download complete
e2de96513ba9: Pull complete
745dfb2690dd: Pull complete
64c8194480fe: Pull complete
9a9a644fdd6a: Pull complete
6c53d0b2a666: Pull complete
e76228b47809: Pull complete
d9aae54b5831: Pull complete
e72112c14215: Pull complete
Digest: sha256:df221db836e1754089190208cee7eeda94f233197056426eda74a43ab1abeac2
Status: Downloaded newer image for nginx:alpine
docker.io/library/nginx:alpine
ayushpatel@docker-lab:~/docker-lab$ docker pull alpine:latest
latest: Pulling from library/alpine
797dd00a0fc7: Download complete
6d0606d1815c: Download complete
Digest: sha256:294b683cb724975bec92580e1e685676bd4b50bda910ddb8c51d4cabeaec77e6
Status: Downloaded newer image for alpine:latest
docker.io/library/alpine:latest
ayushpatel@docker-lab:~/docker-lab$ docker images
IMAGE          ID              DISK USAGE  CONTENT SIZE  EXTRA
alpine:latest  294b683cb724    13MB        3.94MB
hello-world:latest  5e2309035332    25.9kB      9.49kB        U
nginx:alpine    df221db836e1    94.4MB      27.2MB
ayushpatel@docker-lab:~/docker-lab$
```

## Step 4: Run a command inside a container

| Command                                                                                              | Purpose                                                                                                                                                                                               |
|------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>docker run --rm alpine cat /etc/os-release</code>                                              | Starts an Alpine container, runs one command inside it (shows the container's OS) and removes the container afterwards (--rm).                                                                        |
| <code>docker run --rm alpine sh -c 'echo Hello from Ayush Patel, Roll No 202301084; hostname'</code> | Runs a small shell script inside a container. The hostname printed is the container ID, showing each container is an isolated environment. (Interactive form: <code>docker run -it alpine sh</code> ) |

```
ayushpatel@docker-lab: ~/docker-lab - Step 4: Run a command inside a container

ayushpatel@docker-lab:~/docker-lab$ docker run --rm alpine cat /etc/os-release
NAME="Alpine Linux"
ID=alpine
VERSION_ID=3.24.2
PRETTY_NAME="Alpine Linux v3.24"
HOME_URL="https://alpinelinux.org/"
BUG_REPORT_URL="https://gitlab.alpinelinux.org/alpine/aports/-/issues"
ayushpatel@docker-lab:~/docker-lab$ docker run --rm alpine sh -c 'echo Hello from Ayush Patel, Roll No 202301084; hostname'
Hello from Ayush Patel, Roll No 202301084
16a9108abc3b
ayushpatel@docker-lab:~/docker-lab$
```

## Step 5: Run a container in background with port mapping

| Command                                                                    | Purpose                                                                                                                      |
|----------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <code>docker run -d --name ayushpatel-nginx -p 8080:80 nginx:alpine</code> | Starts Nginx in detached mode (-d, background), names it ayushpatel-nginx and maps host port 8080 to container port 80 (-p). |
| <code>docker ps</code>                                                     | Lists running containers with ID, image, status and port mappings.                                                           |
| <code>curl -s http://localhost:8080   grep -i '&lt;title&gt;'</code>       | Sends an HTTP request to the container through the mapped port to prove the web server is reachable.                         |

```
ayushpatel@docker-lab: ~/docker-lab - Step 5: Run a container in background with port mapping

ayushpatel@docker-lab:~/docker-lab$ docker run -d --name ayushpatel-nginx -p 8080:80 nginx:alpine
67537d70112d0b339168bc93fcedacc0d9321a4bc6d57e9154982566cb85137
ayushpatel@docker-lab:~/docker-lab$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
67537d70112    nginx:alpine   "/docker-entrypoint...." 2 seconds ago  Up 2 seconds  0.0.0.0:8080->80/tcp               ayushpatel-nginx
ayushpatel@docker-lab:~/docker-lab$ curl -s http://localhost:8080 | grep -i '<title>'
<title>Welcome to nginx!</title>
ayushpatel@docker-lab:~/docker-lab$
```

## Step 6: Logs, exec and inspect

| Command                                                                                                                                                     | Purpose                                                                                                 |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| <code>docker logs ayushpatel-nginx 2&gt;&amp;1   tail -n 4</code>                                                                                           | Shows the output/logs written by the container (here the last 4 lines, including our curl request).     |
| <code>docker exec ayushpatel-nginx ls -l /usr/share/nginx/html</code>                                                                                       | Runs a command inside the already-running container (docker exec) - here listing Nginx's web folder.    |
| <code>docker inspect --format 'Name={{.Name}} IP={{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}} Status={{.State.Status}}' ayushpatel-nginx</code> | Shows low-level details of the container (name, internal IP address, state) using a Go template filter. |

```
ayushpatel@docker-lab: ~/docker-lab - Step 6: Logs, exec and inspect

ayushpatel@docker-lab:~/docker-lab$ docker logs ayushpatel-nginx 2>&1 | tail -n 4
2026/10/03 17:11:27 [notice] 1#1: start worker process 21
2026/10/03 17:11:27 [notice] 1#1: start worker process 22
2026/10/03 17:11:27 [notice] 1#1: start worker process 23
172.17.0.1 - - [03/Oct/2026:17:11:31 +0000] "GET / HTTP/1.1" 200 896 "-" "curl/8.5.0" "-"
ayushpatel@docker-lab:~/docker-lab$ docker exec ayushpatel-nginx ls -l /usr/share/nginx/html
total 8
-rw-r--r-- 1 root root 497 Sep 15 14:18 50x.html
-rw-r--r-- 1 root root 896 Sep 15 14:18 index.html
ayushpatel@docker-lab:~/docker-lab$ docker inspect --format 'Name={{.Name}} IP={{range \
> .NetworkSettings.Networks}}{{.IPAddress}}{{end}} Status={{.State.Status}}' ayushpatel-nginx
Name=/ayushpatel-nginx IP=172.17.0.2 Status=running
ayushpatel@docker-lab:~/docker-lab$
```

## Step 7: Container lifecycle: stop, start, remove

| Command                                                  | Purpose                                                                 |
|----------------------------------------------------------|-------------------------------------------------------------------------|
| <code>docker stop ayushpatel-nginx</code>                | Gracefully stops the running container.                                 |
| <code>docker ps -a --filter name=ayushpatel-nginx</code> | Lists all containers including stopped ones (-a); status is now Exited. |
| <code>docker start ayushpatel-nginx</code>               | Starts the stopped container again (same container, same settings).     |
| <code>docker ps --filter name=ayushpatel-nginx</code>    | Confirms the container is Up again.                                     |
| <code>docker rm -f ayushpatel-nginx</code>               | Force-removes the container (stops it first because of -f).             |

```
ayushpatel@docker-lab: ~/docker-lab - Step 7: Container lifecycle: stop, start, remove

ayushpatel@docker-lab:~/docker-lab$ docker stop ayushpatel-nginx
ayushpatel-nginx
ayushpatel@docker-lab:~/docker-lab$ docker ps -a --filter name=ayushpatel-nginx
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS          PORTS          NAMES
67537d701112   nginx:alpine  "/docker-entrypoint...."  16 seconds ago  Exited (0) 2 seconds ago                ayushpatel-nginx
ayushpatel@docker-lab:~/docker-lab$ docker start ayushpatel-nginx
ayushpatel-nginx
ayushpatel@docker-lab:~/docker-lab$ docker ps --filter name=ayushpatel-nginx
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS          PORTS          NAMES
67537d701112   nginx:alpine  "/docker-entrypoint...."  20 seconds ago  Up 2 seconds    0.0.0.0:8080->80/tcp                ayushpatel-nginx
ayushpatel@docker-lab:~/docker-lab$ docker rm -f ayushpatel-nginx
ayushpatel-nginx
ayushpatel@docker-lab:~/docker-lab$
```

## Step 8: Write a Dockerfile

| Command                     | Purpose                                                                                                                                             |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>ls -R</code>          | Shows the project files: the Dockerfile, the web page in app/, and the Compose file.                                                                |
| <code>cat Dockerfile</code> | Displays the Dockerfile. FROM picks the base image, LABEL adds metadata, COPY adds our page, EXPOSE documents the port, CMD sets the start command. |

```
ayushpatel@docker-lab: ~/docker-lab - Step 8: Write a Dockerfile

ayushpatel@docker-lab:~/docker-lab$ ls -R
.:
Dockerfile
app
docker-compose.yml

./app:
index.html
ayushpatel@docker-lab:~/docker-lab$ cat Dockerfile
# Base image: lightweight Nginx web server on Alpine Linux
FROM nginx:alpine

# Metadata about the image author
LABEL maintainer="Ayush Patel (202301084)"

# Copy our web page into Nginx's default web root
COPY app/index.html /usr/share/nginx/html/index.html

# Document that the container listens on port 80
EXPOSE 80

# Start Nginx in the foreground so the container keeps running
CMD ["nginx", "-g", "daemon off;"]
ayushpatel@docker-lab:~/docker-lab$
```

## Step 9: Build a custom image

| Command                                                               | Purpose                                                                                                             |
|-----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| <code>docker build -t ayushpatel/docker-lab-web:1.0 .</code>          | Builds a new image from the Dockerfile in the current folder (.) and tags it (-t) as ayushpatel/docker-lab-web:1.0. |
| <code>docker images ayushpatel/docker-lab-web</code>                  | Lists the newly built image.                                                                                        |
| <code>docker history ayushpatel/docker-lab-web:1.0   head -n 6</code> | Shows the layers of the image; the top layers correspond to our Dockerfile instructions.                            |

```
ayushpatel@docker-lab: ~/docker-lab - Step 9: Build a custom image

ayushpatel@docker-lab:~/docker-lab$ docker build -t ayushpatel/docker-lab-web:1.0 .
#0 building with "default" instance using docker driver

#1 [internal] load build definition from Dockerfile
#1 transferring dockerfile: 457B done
#1 DONE 0.0s

#2 [internal] load metadata for docker.io/library/nginx:alpine
#2 DONE 0.0s

#3 [internal] load .dockerignore
#3 transferring context: 69B done
#3 DONE 0.0s

#4 [internal] load build context
#4 transferring context: 772B done
#4 DONE 0.0s

#5 [1/2] FROM docker.io/library/nginx:alpine@sha256:df221db836e1754089190208cee7eeda94f233197056426eda74a43ab1abeac2
#5 resolve docker.io/library/nginx:alpine@sha256:df221db836e1754089190208cee7eeda94f233197056426eda74a43ab1abeac2 0.0s
#5 DONE 0.1s

#6 [2/2] COPY app/index.html /usr/share/nginx/html/index.html
#6 DONE 0.0s

#7 exporting to image
#7 exporting layers 0.1s done
#7 exporting manifest sha256:3df15199b9fc76b1db9e245cc78f26dc9bfb8ecab5a423aa1775194b8d8c0287 done
#7 exporting config sha256:165040795d1cae58b9b536df2775fb491f9d2169c22d745f1ef08fd488337008 done
#7 exporting attestation manifest sha256:1a0cd188d6ab91e9b2191f2beedbc4ab614ce894c6245d322b83b3205234a9e9 0.0s done
#7 exporting manifest list sha256:27eb71ecc9864d0f53fa57704aa94dfacbebcf390f038930bf255e16b33b1822
#7 exporting manifest list sha256:27eb71ecc9864d0f53fa57704aa94dfacbebcf390f038930bf255e16b33b1822 done
#7 naming to docker.io/ayushpatel/docker-lab-web:1.0 done
#7 unpacking to docker.io/ayushpatel/docker-lab-web:1.0 0.0s done
#7 DONE 0.2s
ayushpatel@docker-lab:~/docker-lab$ docker images ayushpatel/docker-lab-web
IMAGE                                ID                                  DISK USAGE  CONTENT SIZE  EXTRA
ayushpatel/docker-lab-web:1.0       27eb71ecc986                       93.6MB      26.3MB
ayushpatel@docker-lab:~/docker-lab$ docker history ayushpatel/docker-lab-web:1.0 | head -n 6
IMAGE      CREATED      CREATED BY                                      SIZE      COMMENT
27eb71ecc986  4 seconds ago  CMD ["nginx" "-g" "daemon off;"]              0B        buildkit.dockerfile.v0
<missing>    4 seconds ago  EXPOSE [80/tcp]                                0B        buildkit.dockerfile.v0
<missing>    4 seconds ago  COPY app/index.html /usr/share/nginx/html/in... 24.6kB    buildkit.dockerfile.v0
<missing>    4 seconds ago  LABEL maintainer=Ayush Patel (202301084)        0B        buildkit.dockerfile.v0
<missing>    10 days ago   RUN /bin/sh -c set -x && apkArch="$(cat ...    52.4MB    buildkit.dockerfile.v0
ayushpatel@docker-lab:~/docker-lab$
```

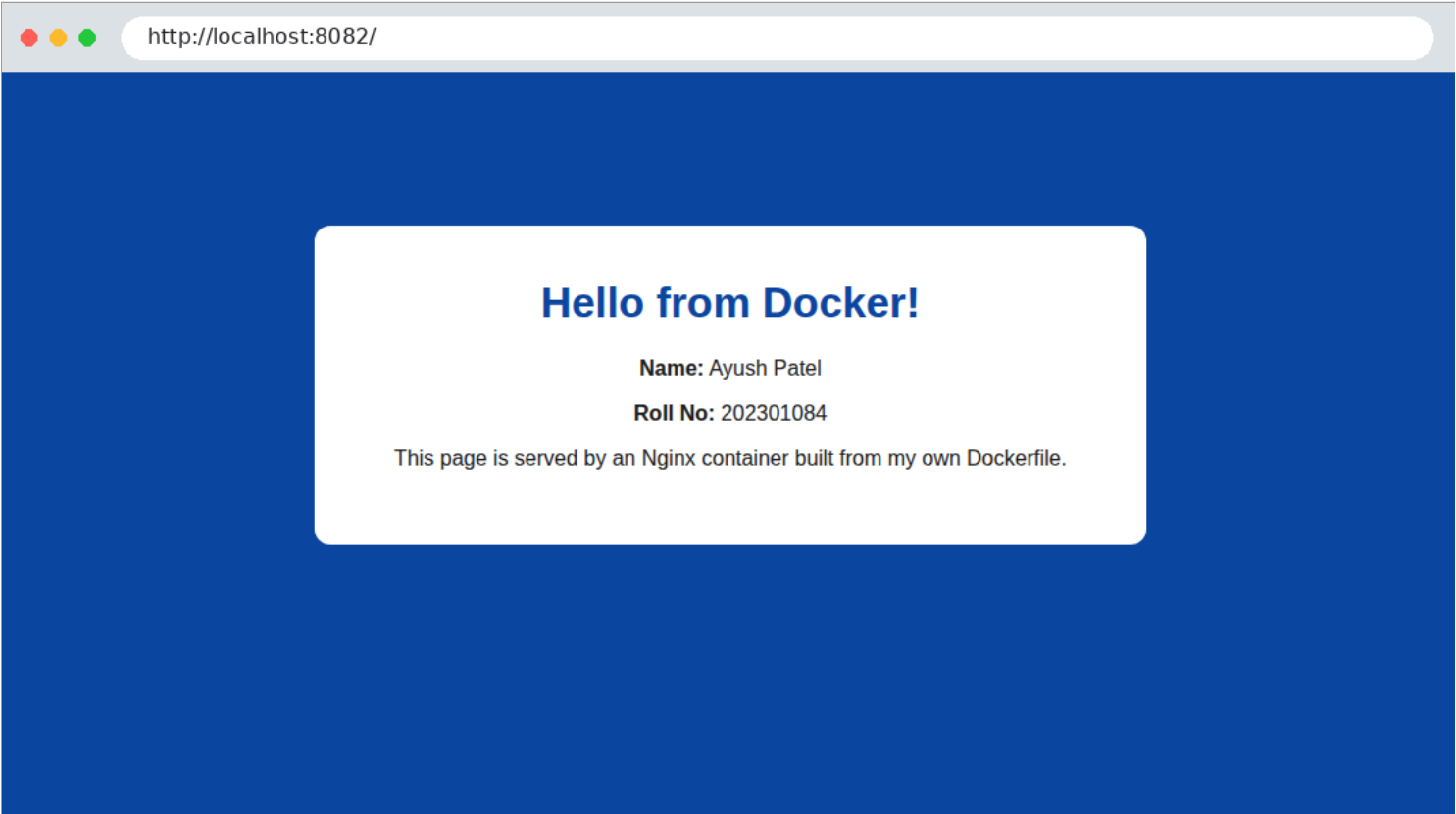
## Step 10: Run the custom image (deployment)

| Command                                                                                   | Purpose                                                                                                 |
|-------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| <code>docker run -d --name ayushpatel-web -p 8082:80 ayushpatel/docker-lab-web:1.0</code> | Deploys our own image as a container on host port 8082.                                                 |
| <code>docker ps --filter name=ayushpatel-web</code>                                       | Confirms the container is running.                                                                      |
| <code>curl -s http://localhost:8082   grep -E 'Name Roll'</code>                          | Fetches the page from the container - it shows my name and roll number, proving the custom image works. |

```
ayushpatel@docker-lab: ~/docker-lab - Step 10: Run the custom image (deployment)

ayushpatel@docker-lab:~/docker-lab$ docker run -d --name ayushpatel-web -p 8082:80 ayushpatel/docker-lab-web:1.0
5f83be4e3030cfc85b2e3e875f577b4ba0c70bb5d6d2abce50e8086d49f029a9
ayushpatel@docker-lab:~/docker-lab$ docker ps --filter name=ayushpatel-web
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
5f83be4e3030   ayushpatel/docker-lab-web:1.0       "/docker-entrypoint..." 2 seconds ago  Up 2 seconds  0.0.0.0:8082->80/tcp
ayushpatel@docker-lab:~/docker-lab$ curl -s http://localhost:8082 | grep -E 'Name|Roll'
<p><b>Name:</b> Ayush Patel</p>
<p><b>Roll No:</b> 202301084</p>
ayushpatel@docker-lab:~/docker-lab$
```

Browser view of the custom image running with docker run (port 8082).



## Step 11: Named volumes (persistent data)

| Command                                                                                                               | Purpose                                                                                                                   |
|-----------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <code>docker volume create ayush-data</code>                                                                          | Creates a named volume - storage managed by Docker that lives outside any container.                                      |
| <code>docker volume ls</code>                                                                                         | Lists all volumes.                                                                                                        |
| <code>docker run --rm -v ayush-data:/data alpine sh -c "echo 'Ayush Patel - 202301084' &gt; /data/student.txt"</code> | Container #1 mounts the volume at /data and writes a file, then is deleted (--rm).                                        |
| <code>docker run --rm -v ayush-data:/data alpine cat /data/student.txt</code>                                         | Container #2 (a brand new one) mounts the same volume and reads the file - data persisted after container #1 was removed. |
| <code>docker volume inspect ayush-data</code>                                                                         | Shows volume details such as where Docker stores it on the host (Mountpoint).                                             |

```
ayushpatel@docker-lab: ~/docker-lab - Step 11: Named volumes (persistent data)

ayushpatel@docker-lab:~/docker-lab$ docker volume create ayush-data
ayush-data
ayushpatel@docker-lab:~/docker-lab$ docker volume ls
DRIVER      VOLUME NAME
local       ayush-data
ayushpatel@docker-lab:~/docker-lab$ docker run --rm -v ayush-data:/data alpine sh -c "echo 'Ayush Patel - 202301084' > \
> /data/student.txt"
ayushpatel@docker-lab:~/docker-lab$ docker run --rm -v ayush-data:/data alpine cat /data/student.txt
Ayush Patel - 202301084
ayushpatel@docker-lab:~/docker-lab$ docker volume inspect ayush-data
[
  {
    "CreatedAt": "2026-10-03T17:12:10Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/ayush-data/_data",
    "Name": "ayush-data",
    "Options": null,
    "Scope": "local"
  }
]
```

## Step 12: Bind mount (host folder into container)

| Command                                                                                                                                         | Purpose                                                                                                    |
|-------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| <code>mkdir -p site &amp;&amp; echo '&lt;h1&gt;Bind mount page by Ayush Patel (202301084)&lt;/h1&gt;' &gt; site/index.html</code>               | Creates a folder on the host with a simple HTML page.                                                      |
| <code>docker run -d --name ayushpatel-bind -p 8083:80 -v "\$(pwd)/site":/usr/share/nginx/html:ro nginx:alpine</code>                            | Runs Nginx and mounts the host folder into the container's web root (read-only, :ro).                      |
| <code>curl -s http://localhost:8083</code>                                                                                                      | The container serves the host file.                                                                        |
| <code>echo '&lt;h1&gt;Updated on the host - no rebuild needed!&lt;/h1&gt;' &gt; site/index.html &amp;&amp; curl -s http://localhost:8083</code> | Edit the file on the host; the container immediately serves the new content because it is the same folder. |

```
ayushpatel@docker-lab: ~/docker-lab - Step 12: Bind mount (host folder into container)

ayushpatel@docker-lab:~/docker-lab$ mkdir -p site && echo '<h1>Bind mount page by Ayush Patel (202301084)</h1>' > \
> site/index.html
ayushpatel@docker-lab:~/docker-lab$ docker run -d --name ayushpatel-bind -p 8083:80 -v "$(pwd)/site":/usr/share/nginx/html:ro \
> nginx:alpine
425bfb6d57f9b6373ffecc9fc65aa7bb2dbb46df8985291c6bd260882bbb7ee8
ayushpatel@docker-lab:~/docker-lab$ curl -s http://localhost:8083
<h1>Bind mount page by Ayush Patel (202301084)</h1>
ayushpatel@docker-lab:~/docker-lab$ echo '<h1>Updated on the host - no rebuild needed!</h1>' > site/index.html && curl -s \
> http://localhost:8083
<h1>Updated on the host - no rebuild needed!</h1>
ayushpatel@docker-lab:~/docker-lab$
```

## Step 13: Docker networks

| Command                                                                                                         | Purpose                                                                                            |
|-----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| <code>docker network create ayush-net</code>                                                                    | Creates a user-defined bridge network so containers can talk to each other by name.                |
| <code>docker run -d --name ayush-server --network ayush-net nginx:alpine</code>                                 | Starts a web server container attached to the network.                                             |
| <code>docker run --rm --network ayush-net alpine wget -qO- http://ayush-server   grep -i '&lt;title&gt;'</code> | A second container on the same network reaches the server using its container name (built-in DNS). |
| <code>docker network ls</code>                                                                                  | Lists networks (bridge, host, none and our ayush-net).                                             |

```
ayushpatel@docker-lab: ~/docker-lab  -  Step 13: Docker networks

ayushpatel@docker-lab:~/docker-lab$ docker network create ayush-net
8fa69a67150f5b3624e02102a285cf6fa45e0587e5bbc6ae999fea7942d0188d
ayushpatel@docker-lab:~/docker-lab$ docker run -d --name ayush-server --network ayush-net nginx:alpine
dba03ae596d2215929436576e61c096cd12f4996c1a83af82e53cdc132882e21
ayushpatel@docker-lab:~/docker-lab$ docker run --rm --network ayush-net alpine wget -qO- http://ayush-server | grep -i '<title>'
<title>Welcome to nginx!</title>
ayushpatel@docker-lab:~/docker-lab$ docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
8fa69a67150f        ayush-net           bridge              local
64d61954f294        bridge              bridge              local
f4eee35c36b5        host                host                local
62647d20a688        none                null                local
ayushpatel@docker-lab:~/docker-lab$
```

## Step 14: Tag and save an image

| Command                                                                                                           | Purpose                                                                                                                             |
|-------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <code>docker tag ayushpatel/docker-lab-web:1.0 ayushpatel/docker-lab-web:latest</code>                            | Adds another tag (name) to the same image - this is how images are prepared for pushing to a registry (docker login + docker push). |
| <code>docker images ayushpatel/docker-lab-web</code>                                                              | Both tags point to the same IMAGE ID.                                                                                               |
| <code>docker save -o docker-lab-web.tar ayushpatel/docker-lab-web:1.0 &amp;&amp; ls -lh docker-lab-web.tar</code> | Exports the image to a .tar file so it can be copied to another machine and loaded with docker load.                                |

```
ayushpatel@docker-lab: ~/docker-lab - Step 14: Tag and save an image

ayushpatel@docker-lab:~/docker-lab$ docker tag ayushpatel/docker-lab-web:1.0 ayushpatel/docker-lab-web:latest
ayushpatel@docker-lab:~/docker-lab$ docker images ayushpatel/docker-lab-web
IMAGE                                ID                                DISK USAGE  CONTENT SIZE  EXTRA
ayushpatel/docker-lab-web:1.0        27eb71ecc986                      93.6MB       26.3MB        U
ayushpatel/docker-lab-web:latest      27eb71ecc986                      93.6MB       26.3MB        U
ayushpatel@docker-lab:~/docker-lab$ docker save -o docker-lab-web.tar ayushpatel/docker-lab-web:1.0 && ls -lh docker-lab-web.tar
-rw-r----- 1 ayushpatel ayushpatel 26M Oct  3 17:12 docker-lab-web.tar
ayushpatel@docker-lab:~/docker-lab$
```

## Step 15: Deploy with Docker Compose

---

| Command                                                          | Purpose                                                                                                      |
|------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| <code>cat docker-compose.yml</code>                              | Shows the Compose file: one service built from our Dockerfile, port 8081, and a named volume for Nginx logs. |
| <code>docker compose up -d --build</code>                        | Builds the image and starts all services in the background with a single command.                            |
| <code>docker compose ps</code>                                   | Lists the containers managed by this Compose project.                                                        |
| <code>curl -s http://localhost:8081   grep -E 'Name Roll'</code> | Verifies the Compose deployment is serving the page.                                                         |

```
ayushpatel@docker-lab:~/docker-lab$ cat docker-compose.yml
services:
  web:
    build: .
    image: ayushpatel/docker-lab-web:1.0
    container_name: ayushpatel-compose-web
    ports:
      - "8081:80"
    volumes:
      - web-logs:/var/log/nginx
    restart: unless-stopped

volumes:
  web-logs:
```

```
ayushpatel@docker-lab:~/docker-lab$ docker compose up -d --build
#1 [internal] load local bake definitions
#1 reading from stdin 515B done
#1 DONE 0.0s

#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile: 457B done
#2 DONE 0.0s

#3 [internal] load metadata for docker.io/library/nginx:alpine
#3 DONE 0.0s

#4 [internal] load .dockerignore
#4 transferring context: 69B done
#4 DONE 0.0s

#5 [internal] load build context
#5 transferring context: 59B done
#5 DONE 0.0s

#6 [1/2] FROM docker.io/library/nginx:alpine@sha256:df221db836e1754089190208cee7eeda94f233197056426eda74a43ab1abeac2
#6 resolve docker.io/library/nginx:alpine@sha256:df221db836e1754089190208cee7eeda94f233197056426eda74a43ab1abeac2 0.0s done
... (5 lines trimmed) ...
#8 exporting to image
#8 exporting layers done
#8 exporting manifest sha256:de47fb02a8730260567bef3bbfa36e375924000e4e679531a7441ac964e7460a done
#8 exporting config sha256:d9ec8d2e24d0ba138d091a6e5b485c2f67b8f9e1c45ee5ae44eeeeeef3c28a049 done
#8 exporting attestation manifest sha256:9b58ca490ddabb1fdd1c247ef4dd8f5e65fb7af02fce68da73873c1a19229deb 0.0s done
#8 exporting manifest list sha256:23f032642132e1b403f7d274f2eb443578dcbc554ebb3c2a53f4a15696650032 done
#8 naming to docker.io/ayushpatel/docker-lab-web:1.0 done
#8 unpacking to docker.io/ayushpatel/docker-lab-web:1.0 0.0s done
#8 DONE 0.1s

#9 resolving provenance for metadata file
#9 DONE 0.0s
Image ayushpatel/docker-lab-web:1.0 Building
Image ayushpatel/docker-lab-web:1.0 Built
Network docker-lab_default Creating
Network docker-lab_default Created
Volume docker-lab_web-logs Creating
Volume docker-lab_web-logs Created
Container ayushpatel-compose-web Creating
Container ayushpatel-compose-web Created
Container ayushpatel-compose-web Starting
Container ayushpatel-compose-web Started
ayushpatel@docker-lab:~/docker-lab$ docker compose ps
NAME                                IMAGE                                COMMAND                                SERVICE    CREATED        STATUS
ayushpatel-compose-web             ayushpatel/docker-lab-web:1.0      "/docker-entrypoint..."            web        2 seconds ago Up 2
seconds    0.0.0.0:8081->80/tcp
ayushpatel@docker-lab:~/docker-lab$ curl -s http://localhost:8081 | grep -E 'Name|Roll'
<p><b>Name:</b> Ayush Patel</p>
<p><b>Roll No:</b> 202301084</p>
ayushpatel@docker-lab:~/docker-lab$
```

Browser view of the same app deployed with Docker Compose (port 8081).



http://localhost:8081/

# Hello from Docker!

**Name:** Ayush Patel

**Roll No:** 202301084

This page is served by an Nginx container built from my own Dockerfile.

## Step 16: Monitoring: stats and compose logs

| Command                                                                                       | Purpose                                                                  |
|-----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| <code>docker stats --no-stream --format 'table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}'</code> | Shows a one-time snapshot of CPU and memory usage of running containers. |
| <code>docker compose logs --tail 3</code>                                                     | Shows the last log lines of the Compose services.                        |
| <code>docker system df</code>                                                                 | Shows how much disk space images, containers and volumes use.            |

```
ayushpatel@docker-lab: ~/docker-lab - Step 16: Monitoring: stats and compose logs

ayushpatel@docker-lab:~/docker-lab$ docker stats --no-stream --format 'table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}'
NAME                CPU %    MEM USAGE / LIMIT
ayushpatel-compose-web 0.00%    5.137MiB / 15.72GiB
ayush-server          0.00%    4.617MiB / 15.72GiB
ayushpatel-bind        0.00%    4.582MiB / 15.72GiB
ayushpatel-web         0.00%    4.617MiB / 15.72GiB
ayushpatel@docker-lab:~/docker-lab$ docker compose logs --tail 3
ayushpatel-compose-web | 2026/10/03 17:12:51 [notice] 1#1: start worker process 22
ayushpatel-compose-web | 2026/10/03 17:12:51 [notice] 1#1: start worker process 23
ayushpatel-compose-web | 172.19.0.1 - - [03/Oct/2026:17:12:56 +0000] "GET / HTTP/1.1" 200 702 "-" "curl/8.5.0" "-"
ayushpatel@docker-lab:~/docker-lab$ docker system df
TYPE                TOTAL    ACTIVE    SIZE      RECLAIMABLE
Images              5         4      94.64MB    88.02kB (0%)
Containers          5         4      217.1kB    4.096kB (1%)
Local Volumes       2         1        46B       24B (52%)
Build Cache        12         0     26.38MB    28.67kB
ayushpatel@docker-lab:~/docker-lab$
```

## Step 17: Clean up

| Command                                                                         | Purpose                                                            |
|---------------------------------------------------------------------------------|--------------------------------------------------------------------|
| <code>docker compose down -v</code>                                             | Stops and removes the Compose containers, network and volume (-v). |
| <code>docker rm -f ayushpatel-web ayushpatel-bind ayush-server</code>           | Removes the remaining containers.                                  |
| <code>docker network rm ayush-net &amp;&amp; docker volume rm ayush-data</code> | Removes the custom network and volume.                             |
| <code>docker container prune -f</code>                                          | Deletes all stopped containers (e.g. the hello-world one).         |
| <code>docker ps -a</code>                                                       | Confirms no containers are left.                                   |

```
ayushpatel@docker-lab: ~/docker-lab - Step 17: Clean up

ayushpatel@docker-lab:~/docker-lab$ docker compose down -v
Container ayushpatel-compose-web Stopping
Container ayushpatel-compose-web Stopped
Container ayushpatel-compose-web Removing
Container ayushpatel-compose-web Removed
Network docker-lab_default Removing
Volume docker-lab_web-logs Removing
Volume docker-lab_web-logs Removed
Network docker-lab_default Removed
ayushpatel@docker-lab:~/docker-lab$ docker rm -f ayushpatel-web ayushpatel-bind ayush-server
ayushpatel-web
ayushpatel-bind
ayush-server
ayushpatel@docker-lab:~/docker-lab$ docker network rm ayush-net && docker volume rm ayush-data
ayush-net
ayush-data
ayushpatel@docker-lab:~/docker-lab$ docker container prune -f
Deleted Containers:
16466151dbdd353cd05f20646176b73ae02dc27b4a0a7f2119521e63672dc51b

Total reclaimed space: 4.096kB
ayushpatel@docker-lab:~/docker-lab$ docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS        NAMES
ayushpatel@docker-lab:~/docker-lab$
```

## Conclusion

---

In this lab I installed and verified Docker, ran containers from public images, managed the container lifecycle, wrote a Dockerfile and built my own image, used named volumes and bind mounts for data, connected containers with a custom network, tagged/saved the image, and deployed the application with Docker Compose. The browser screenshots confirm the application ran successfully.